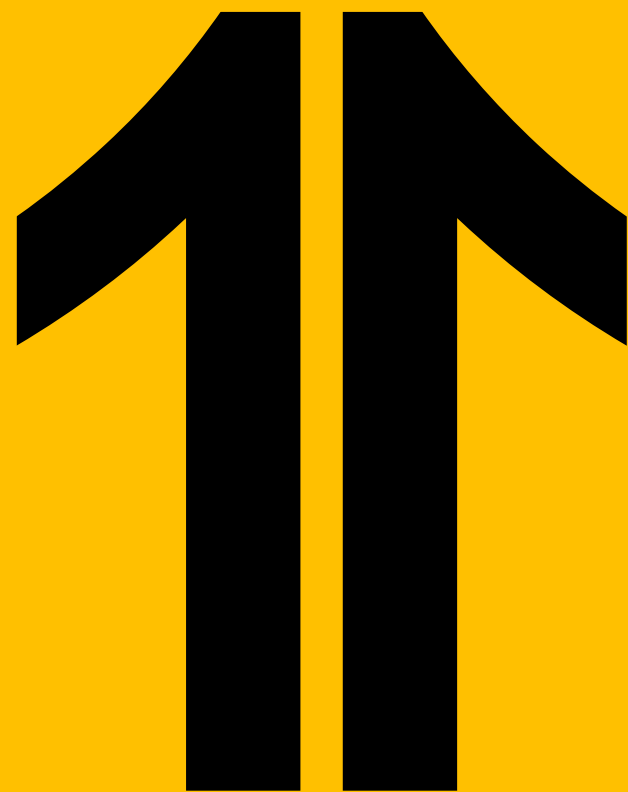




UPN

**UNIVERSIDAD
PRIVADA
DEL NORTE**



FUNDAMENTOS DE ALGORITMOS



ARREGLOS BIDIMENSIONALES

SEMANA 15

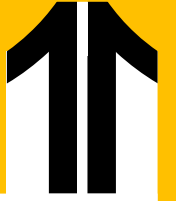
AGENDA



III	Arreglos Al finalizar la unidad, el estudiante implementa algoritmos para resolver problemas utilizando métodos de caso, formularios y arreglos unidimensionales y bidimensionales en el lenguaje C#; demostrando lógica y optimización de código en sus programas, y aplicando un sistema de control de versiones para gestionar su desarrollo y facilitar la colaboración.	11	Arreglos unidimensionales: Operaciones: Registrar, mostrar, buscar, modificar (D. FLUJO o pseudocódigo y código)
		12	Arreglos unidimensionales Operaciones: Insertar, eliminar, ordenar (burbuja) (D. FLUJO o pseudocódigo y código)
		13	Taller de desarrollo de casos Evaluación T3
		14	Cadenas de Caracteres. Funciones de manipulación de cadenas de caracteres Arreglo de cadenas. Arreglos paralelos. Ordenamientos. (D. FLUJO o pseudocódigo y código).
		15	Arreglos bidimensionales (Matrices) Operaciones con matrices (D. FLUJO o pseudocódigo y código)
		16	EVALUACIÓN FINAL



Al término de la clase, el estudiante elabora matrices usando códigos en C# mostrando una lógica coherente al implementar su algoritmo.



ARREGLOS BIDIMENSIONALES

C# permite arreglos bidimensionales de dos tipos:

- ✓ Multidimensional: Es una matriz rectangular de datos.
- ✓ Irregular (Jagged):): Es un arreglo de arreglos por lo que cada fila puede tener longitud diferente. Los arreglos en C# son de este tipo.



ARREGLOS BIDIMENSIONALES

Declaración

```
int[,] array = new int[4, 2];
```

```
int[, ,] array1 = new int[4, 2, 3];
```

```
array5[2, 1] = 25;
```

```
int elementValue = array5[2, 1];
```

```
int[,] array6 = new int[10, 10];
```

```
// Two-dimensional array.
int[,] array2D = new int[,] { { 1, 2 }, { 3, 4 }, { 5, 6 }, { 7, 8 } };
// The same array with dimensions specified.
int[,] array2Da = new int[4, 2] { { 1, 2 }, { 3, 4 }, { 5, 6 }, { 7, 8 } };
// A similar array with string elements.
string[,] array2Db = new string[3, 2] { { "one", "two" }, { "three", "four" },
                                         { "five", "six" } };

// Three-dimensional array.
int[, ,] array3D = new int[, ,] { { { 1, 2, 3 }, { 4, 5, 6 } },
                                   { { 7, 8, 9 }, { 10, 11, 12 } } };
// The same array with dimensions specified.
int[, ,] array3Da = new int[2, 2, 3] { { { 1, 2, 3 }, { 4, 5, 6 } },
                                         { { 7, 8, 9 }, { 10, 11, 12 } } };

// Accessing array elements.
System.Console.WriteLine(array2D[0, 0]);
System.Console.WriteLine(array2D[0, 1]);
System.Console.WriteLine(array2D[1, 0]);
System.Console.WriteLine(array2D[1, 1]);
System.Console.WriteLine(array2D[3, 0]);
System.Console.WriteLine(array2Db[1, 0]);
System.Console.WriteLine(array3Da[1, 0, 1]);
System.Console.WriteLine(array3D[1, 1, 2]);

// Getting the total count of elements or the length of a given dimension.
var allLength = array3D.Length;
var total = 1;
for (int i = 0; i < array3D.Rank; i++)
{
    total *= array3D.GetLength(i);
}
System.Console.WriteLine("{0} equals {1}", allLength, total);

// Output:
// 1
// 2
// 3
// 4
// 7
// three
// 8
// 12
```



ARREGLOS BIDIMENSIONALES

Ver información del arreglo

```
int[] arr = {1,2,4};  
int[,] mulArr = new int[2,3];  
Console.WriteLine("Length of arr : {0}", arr.Length); // output: 3  
Console.WriteLine("Length of armulArrrrMul : {0}", mulArr.Length); // output: 6
```

```
Console.WriteLine("Rank of arr : {0}", arr.Rank); // output: 1  
Console.WriteLine("Rank of armulArrrrMul : {0}", mulArr.Rank); // output: 2  
Console.WriteLine("Rank of array : {0}", new string[2,3,3].Rank); // output: 3  
// [2,3,3] = 2 rows(arrays) of 3 arrays, each having 3 elements
```

Rank retorna la cantidad de
dimensiones del arreglo



ARREGLOS BIDIMENSIONALES

Para recorrer sus elementos

```
1  for(int row = 0; row < sections.GetLength(0); row++){  
2      for (int col = 0; col < sections.GetLength(1); col++)  
3      {  
4          char grade = (row == 0) ? 'A' : 'B';  
5          Console.WriteLine("{0} scored grade {1}", sections[row,col], grade); //  
6      }  
7  }
```

```
1  foreach (string student in sections)  
2  {  
3      Console.WriteLine("{0}", student);  
4  }
```



ARREGLOS BIDIMENSIONALES

Declaración

```
int[][] jaggedArray = new int[3][];
```

```
jaggedArray[0] = new int[5];  
jaggedArray[1] = new int[4];  
jaggedArray[2] = new int[2];
```

```
jaggedArray[0] = new int[] { 1, 3, 5, 7, 9 };  
jaggedArray[1] = new int[] { 0, 2, 4, 6 };  
jaggedArray[2] = new int[] { 11, 22 };
```

```
int[][] jaggedArray2 = new int[][]  
{  
    new int[] { 1, 3, 5, 7, 9 },  
    new int[] { 0, 2, 4, 6 },  
    new int[] { 11, 22 }  
};
```

```
int[][] jaggedArray3 =  
{  
    new int[] { 1, 3, 5, 7, 9 },  
    new int[] { 0, 2, 4, 6 },  
    new int[] { 11, 22 }  
};
```



ARREGLOS BIDIMENSIONALES

Asignación y Consulta de valores

```
// Assign 77 to the second element ([1]) of the first array ([0]):  
jaggedArray3[0][1] = 77;  
  
// Assign 88 to the second element ([1]) of the third array ([2]):  
jaggedArray3[2][1] = 88;
```



ARREGLOS BIDIMENSIONALES

Para recorrer sus elementos

```
for (int size = 0; size < enteries.Length; size++)
{
    Console.WriteLine("Team {0}: ",enteries[size][0]); // team name
    for (int innerArrSize = 1; innerArrSize < enteries[size].Length; innerArrSize++)
    {
        Console.WriteLine(" {0}",enteries[size][innerArrSize]); // names of team members
    }
}
```

```
1  foreach (string[] team in enteries) // string[]: for inner array
2  {
3      foreach (string element in team)
4      {
5          Console.WriteLine("{0}",element);
6      }
7  }
8  //or
9  foreach (var team in enteries)
10 {
11     foreach (var element in team)
12     {
13         Console.WriteLine("{0}",element);
14     }
15 }
```



Realizar un programa que permita imprimir la siguiente serie:

```
1 0 1 0 1 0
1 0 1 0 1 0
1 0 1 0 1 0
1 0 1 0 1 0
1 0 1 0 1 0
1 0 1 0 1 0
```



Solución:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;

namespace PruebaVector
{
    class PruebaVector
    {
        private int[,] serie;

        public void Cargar()
        {
            serie = new int[10,10];
            for (int i = 1; i <= 6; i++)
            {
                for (int j = 1; j <= 6; j++)
                {
                    if (j % 2 == 0)
                    {
                        serie[i, j] = 0;
                    }
                    else
                    {
                        serie[i, j] = 1;
                    }
                }
            }
        }
    }
}
```

Solución (continuación):

```
public void visualizar()
{
    for (int i = 1; i <= 5; i++)
    {
        Console.WriteLine("\n");
        for (int j = 1; j <= 6; j++)
        {
            Console.Write(serie[i,j]+" ");
        }
        Console.ReadKey();
    }

    static void Main(string[] args)
    {
        PruebaVector pv = new PruebaVector();
        pv.Cargar();
        pv.visualizar();
    }
}
```

